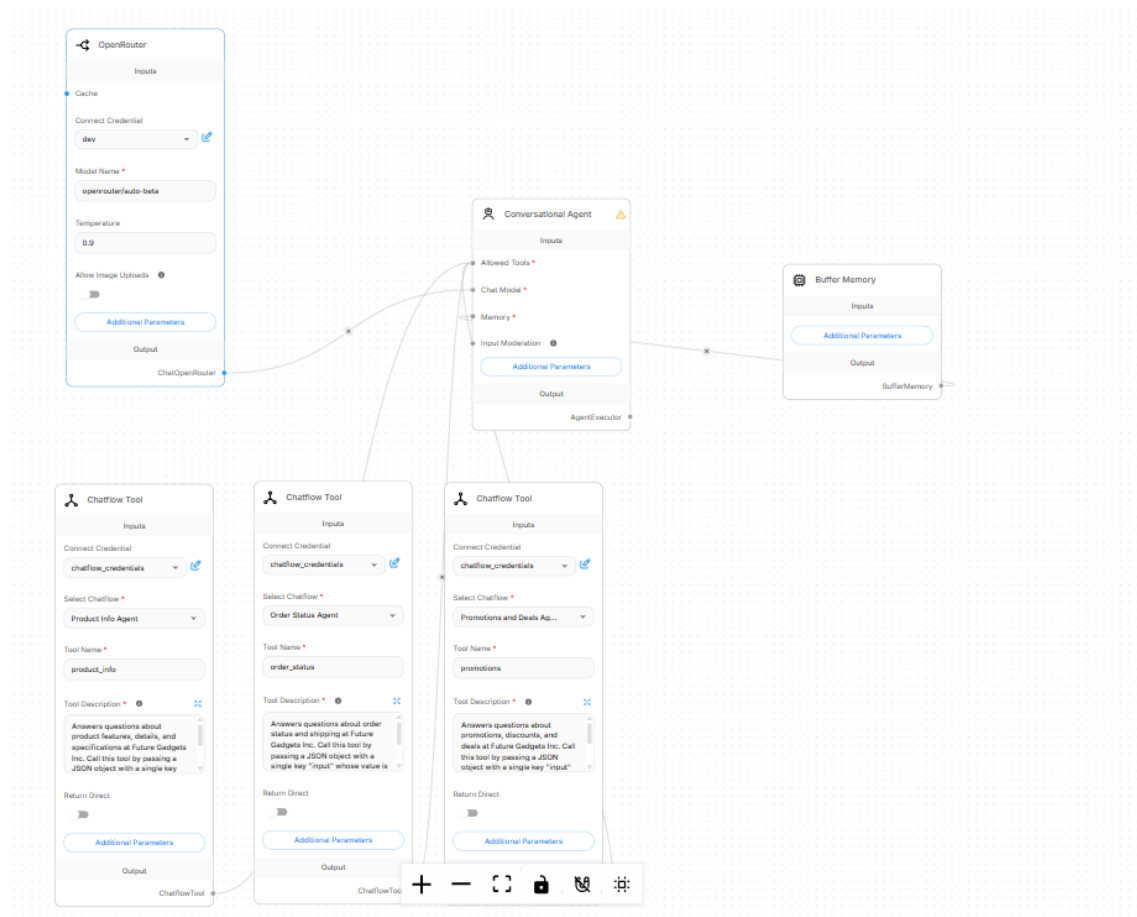


Future Gadgets Inc.

Deliverable 1. Orchestrator agent

The diagram shows the orchestrator agent using a Conversational Agent component due to multi-prompt chains components proposed on the exercise were not available on Flowise AI 3.14.



Deliverable 2. Design methodology

Design Methodology - Future Gadgets Inc. Support Crew

I followed a narrow specialist, wide orchestrator methodology: decompose the support workload into three mutually exclusive domains and route between them with a single intelligent dispatcher.

The Product Info Agent answers questions about product features, specifications, and availability using a fixed catalog. The Order Status Agent handles only order tracking and shipping, asking for the order number before issuing the standard 3-5 business day estimate. The Promotions and Deals Agent covers sales and discounts, broadcasting the smartwatch promotion.

Each specialist is intentionally scoped by its system prompt, and each prompt redirects out-of-scope questions elsewhere, which keeps responses consistent and prevents one agent answering outside its domain.

The orchestrator classifies each customer query and delegates via three Chatflow Tools (product_info, order_status, promotions). The tool descriptions justify the design: each states its domain and instructs the model to pass a JSON input, so the router matches queries to the correct tool and invokes it with well-formed arguments. These descriptive, mutually exclusive definitions directly reduce misrouting and tool-calling errors.

Deliverable 3. Risk and governance analysis

No.	Risk	Mitigation
1	Agent provides incorrect product or order information. The specialists rely on simulated or static data (fixed catalog, standard 3-5 day estimate), so a model hallucination could state the wrong price, availability, or delivery promise that damages customer trust and escalates complaints.	Scope each specialist strictly to its domain prompt, ground answers in the static catalog text, and instruct agents to redirect out-of-scope questions instead of guessing. Add conversational logging so any incorrect response can be traced, reviewed, and corrected promptly.
2	Sensitive customer data is mishandled. The Order Status Agent asks for order numbers, and the orchestrator forwards the full user question to the specialist tools. If chat history or request payloads are stored without controls, personal information could be exposed or misused.	Apply a least-privilege policy: keep order data simulation-only (no real backend), avoid echoing personal details beyond the order number, and configure retention and access controls on the chat logs. Never embed API keys in prompts or code.
3	Agent hallucinates or overstates promotions. An unconstrained sales assistant could invent discounts, dates, or conditions that the store never offered, causing legal or financial exposure.	Hard-code the vetted promotion (20% off smartwatches) directly in the prompt, forbid inventing new offers, and monitor and log conversations to detect and correct overstatements.

Deliverable 4. Reflection and learning insights

Working on this project taught me that a multi-agent system is not just about connecting models, but about carefully designing boundaries, responsibilities, and controls.

The biggest lesson was the value of decomposing a problem into narrow, well-scoped specialists instead of one monolithic prompt. Giving each agent a single domain - product

info, order status, or promotions - made its behavior predictable and easy to debug. The orchestrator then becomes a thin dispatcher that only has to classify intent and delegate, which is a much simpler task than answering everything at once.

I also learned that tool descriptions and prompts are part of the system design, not an afterthought. The quality of the routing depended directly on how clearly, I described each tool's domain and expected input format. When descriptions were ambiguous or overlapped, the router made wrong choices; when they were precise and mutually exclusive, routing became accurate.

Another important insight was about governance. Real data, real orders, and real promotions raise risks that a prototype can hide hallucinated information, mishandled personal data, and overstated offers. Adding mitigations like strict domain scoping, static vetted data, and logging forced me to think about the system beyond "does it work" and consider "is it safe."

Finally, I learned that debugging agent behavior requires observability. Tracing which tool was called and what it returned was essential to find where the pipeline broke.

These insights - decompose responsibilities, design prompts as system components, and govern before scaling - are principles I will apply to future AI projects.